COMPUTATIONAL SCIENCES
WINTER TERM 2026
FREIE UNIVERSITÄT BERLIN

Report

Incompressible Navier Stokes Project

Annie, Bianca, Ceren, Gotkug, Jaza, Louis

Gutachter(in):	Dr. Luca Donati
Semester:	Winter term 2026
Verfasser:	Vorname Nachname
Matrikel-Nr.:	1234567
Adresse:	Straße Nr, PLZ Ort
Email:	mail@mail.de
Telefon:	030-123 456 78
Studienfach:	MSc. Computational Sciences
Abgabetermin:	**01. Januar 2099**

Abstract

TEXT

Contents

List of Figures

List of Tables

1 Introduction

1.0.1 Introduction and Physics of the Problem

The focus of this project is the numerical solution of the incompressible Navier-Stokes equations in a two-dimensional domain subject to a thermal gradient. Unlike many steady-state engineering problems, this project follows a transient simulation approach. This is dictated by the physics of Rayleigh-Benard convection, where our objective is to observe the evolution of "convection cells" from an initial state of rest toward dynamic equilibrium.

The system consists of a fluid confined between two horizontal plates; the bottom plate is heated, while the top plate is kept cold. Based on the Boussinesq approximation, density variations are neglected in all terms **except for the buoyancy term** in the vertical momentum equation ($\beta T e_z$). This coupling allows temperature differences to drive fluid motion. Consequently, we should solve a coupled system of three Partial Differential Equations (PDEs): continuity (mass conservation), Navier-Stokes (momentum), and the energy (heat) equation.

1.0.2 Boundary Conditions

1.0.3 Rayleigh-Benard Convection

Rayleigh-Bénard convection cells can be observed in everyday situations such as a boiling kettle or in atmospheric studies. They form when a fluid (such as water or air) is heated from below, causing it to circulate through its entire depth by rising from the bottom, cooling near the top, and sinking again in a repeating loop.

To describe this phenomenon using the example of a water kettle placed on a heater, as the fluid at the bottom begins to warm it becomes less dense and therefore more buoyant. The warmer fluid rises toward the surface while the cooler and denser fluid sinks downward to replace it. As this process continues, a pattern can be observed forming in the fluid flow. These patterns are known as Bénard cells and are caused by Rayleigh-Bénard convection.

As the warm fluid reaches the surface it spreads outward across the top of the container, while the cooler fluid moves inward and downward along the edges of the cells. Once the fluid reaches these cooler regions it begins to lose heat and sink back down toward the bottom of

the container where it can be reheated. This process creates a continuous circulation loop in the fluid. When the temperature difference across the fluid layer becomes sufficiently large, these convection cells form stable and organized patterns across the surface of the fluid. The center of each cell is typically warmer, while the boundaries between cells are cooler, producing the characteristic cellular pattern.

Rayleigh Number

Whether or not Bénard cells form is determined by a non-dimensional quantity called the Rayleigh number (Ra). The Rayleigh number represents the ratio of buoyancy forces that drive convection to the stabilizing effects of thermal diffusion and viscous damping within the fluid.

$$Ra_x = \frac{g\beta(T_b - T_u)x^3}{\nu\alpha} = Gr_x\, Pr \tag{1.1}$$

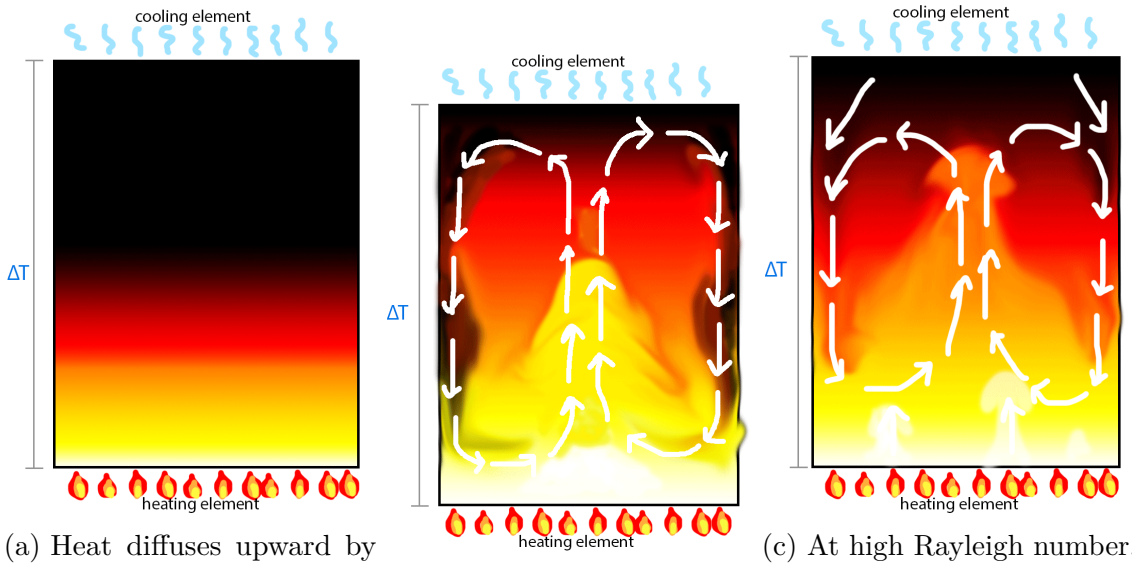
$$Gr_x = \frac{g\beta(T_b - T_u)x^3}{\nu^2} \tag{1.2}$$

$$Pr = \frac{\nu}{\alpha} \tag{1.3}$$

In these expressions, g is the acceleration due to gravity, β is the thermal expansion coefficient of the fluid, ν and α are the fluid kinematic viscosity and thermal diffusivity respectively. The term $(T_b - T_u)$ represents the temperature difference between the bottom and top of the fluid layer, and x (or L) represents the height of the fluid layer.

The Rayleigh number can also be expressed as the product of two other dimensionless numbers: the Grashof number (Gr) and the Prandtl number (Pr). The Grashof number represents the ratio of buoyancy forces to viscous forces in the fluid, while the Prandtl number represents the ratio of momentum diffusion (viscous effects) to thermal diffusion.

For a horizontal fluid layer heated from below, convection begins when the Rayleigh number exceeds a critical value of approximately 1708. If the Rayleigh number is less than 1708 ($Ra < 1708$), heat transfer takes place mainly through thermal conduction, and the fluid layer remains stable with little or no circulation. When the Rayleigh number exceeds 1708 ($Ra > 1708$), the fluid becomes unstable and organized convection cells begin to form, producing the characteristic Rayleigh-Bénard cell pattern. Therefore, low Rayleigh numbers indicate weaker convection and higher Rayleigh numbers indicate stronger, more intense convection.



(a) Heat diffuses upward by conduction. No fluid motion yet.

(b) When $Ra > 1708$, convection begins.

(c) At high Rayleigh number, turbulent convection occurs.

Figure 1.1: **Figure 1. Rayleigh-Bénard Convection (Chang, n.d.)**

1.1 Field of relevance

TEXT

1.2 Research goal

TEXT

1.3 Structure of this report

TEXT

2 Methodology

2.1 First task

2.1.1 Model

2.1.2 Operator Splitting

Consider a typical partial differential equation (PDE) of the form

$$\partial_t \boldsymbol{u} = (A + B + C)\boldsymbol{u}.$$

Here, $\boldsymbol{u}$ denotes the fluid velocity at a given location. The terms A, B, and C are called *operators*, which represent individual processes within the PDE corresponding to different physical effects. As an example, consider the basic advection-diffusion equation

$$\partial_t T = \Delta T - \boldsymbol{u} \cdot \nabla T.$$

Defining the operators $A := \Delta$ and $B := \boldsymbol{u} \cdot \nabla$, the equation can be written as

$$\partial_t T = (A - B)T.$$

Here, A represents the Laplace operator (diffusion), while B represents advection by the fluid velocity field $\boldsymbol{u}$.

For most PDEs, analytical solutions do not exist, and one must instead use numerical methods to approximate their solutions. However, these methods can be computationally expensive. *Operator splitting* reduces this computational cost by partitioning the problem into simpler subproblems, where each operator is treated individually. To continue with our previous example, one solves the following subproblems sequentially:

$$\partial_t \boldsymbol{u} = A\boldsymbol{u} \tag{2.1}$$

$$\partial_t \boldsymbol{u} = B\boldsymbol{u} \tag{2.2}$$

When the first equation is solved numerically, the resulting state is then used as the

initial condition for the second subproblem. This procedure is repeated at each timestep. However, this introduces an approximation error. The intermediate state does not exactly correspond to the solution that would be obtained by analytically solving the combined system. Different operator splitting methods lead to different magnitudes of error. In general, the error remains controlled if the timestep Δt is sufficiently small.

First order Operator Splitting

First-order operator splitting solves each operator sequentially over a timestep Δt. To illustrate this, consider the equation

$$\partial_t \boldsymbol{u} = (A + B)\boldsymbol{u} \tag{2.3}$$

$$\tag{2.4}$$

Instead of solving the full equation directly, the problem is approximated by solving the following subproblems in sequence:

$$\partial_t \boldsymbol{u} = A\boldsymbol{u}, \tag{2.5}$$

$$\partial_t \boldsymbol{u} = B\boldsymbol{u}. \tag{2.6}$$

$$\tag{2.7}$$

For our problem, these operators arise from the advection-diffusion equation, as will be discussed later. Note that applying operator splitting results in two separate **linear** subproblems. This allows us to obtain simple solutions for the resulting split operator problems, namely

$$\boldsymbol{u}_A(t) = e^{(t-t_0)A}\boldsymbol{u}_0 \tag{2.8}$$

$$\boldsymbol{u}_B(t) = e^{(t-t_0)B}\boldsymbol{u}_0 \tag{2.9}$$

In practice, however, we only advance the solution over the next discrete time interval $[t_k, t_{k+1}] = [t_k, t_k + \Delta t]$, rather than solving continuously over the entire time domain. Therefore, the solutions are rewritten for a single timestep as

$$\mathbf{u}_{k+1,A} = e^{(t_{k+1}-t_k)A}\mathbf{u}_{k,A} \tag{2.10}$$

$$\mathbf{u}_{k+1,B} = e^{(t_{k+1}-t_k)B}\mathbf{u}_{k,B} \tag{2.11}$$

Using $\Delta t = t_{k+1} - t_k$, we obtain the timestep formulation

$$\mathbf{u}_{k+1,A} = e^{\Delta t A}\mathbf{u}_{k,A} \tag{2.12}$$

$$\mathbf{u}_{k+1,B} = e^{\Delta t B}\mathbf{u}_{k,B}, \tag{2.13}$$

where an equidistant discretization in time is assumed. For first-order operator splitting the following algorithm is applied to the abstract For first-order operator splitting, the following algorithm is applied to the abstract problem $\partial_t \boldsymbol{u} = (A + B)\boldsymbol{u}$. The equation is split into two separate operator problems:

1. Set $\boldsymbol{u}_{k,A} := \boldsymbol{u}_k$.
2. Solve $\boldsymbol{u}_{k+1,A} = e^{\Delta t A}\boldsymbol{u}_{k,A}$ on the interval $[t_k, t_k + \Delta t]$.
3. Use $\boldsymbol{u}_{k+1,A}$.
4. Consider $\boldsymbol{u}_{k+1,A}$ as the initial condition for the second operator problem by setting $\boldsymbol{u}_{k,B} := \boldsymbol{u}_{k+1,A}$.
5. Solve $\boldsymbol{u}_{k+1,B} = e^{\Delta t B}\boldsymbol{u}_{k,B} = e^{\Delta t B}\boldsymbol{u}_{k+1,A}$.
6. Set $\boldsymbol{u}_{k+1} := \boldsymbol{u}_{k+1,B}$.

Written in a compact form, the procedure becomes

$$\boldsymbol{u}_{k+1} = e^{\Delta t B} e^{\Delta t A} u_k$$

As previously mentioned, this algorithm relies on the assumption that for small timesteps Δt, only small difference remains between $u_{k+1,A}$ and $u_{k+1,B}$ when the operators are applied subsequently. This algorithm is simple and easy to implement. However, the global truncation error (GTE) is of order $\mathcal{O}(\Delta t)$, which motivates the development of second-order splitting methods.

include some computational efficiency stuff, maybe even related to our testing?

include source

Second Order Operator Splitting (Strang-Splitting)

To reduce the GTE of the operator splitting scheme to order $\mathcal{O}(\Delta t^2)$, Strang splitting can be applied. The solution $\boldsymbol{u}_{k+1}$ at the next timestep $t_{k+1} = t_k + \Delta t$ is approximated by

$$\boldsymbol{u}_{k+1} = e^{\frac{\Delta t}{2} A} e^{\Delta t B} e^{\frac{\Delta t}{2} A}\boldsymbol{u}_k \tag{2.14}$$

2.1.3 The Velocity Equation

We have seen operator splitting for a general problem, and now apply the method to the context of the velocity equation from the Navier-Stokes equations

$$\partial_t \boldsymbol{u} + \boldsymbol{u} \cdot \nabla \boldsymbol{u} + \frac{1}{\rho} \nabla p = \nu \Delta \boldsymbol{u} + \beta T \boldsymbol{e}_z$$

This equation can be split into three different operators:

1. Advection: $\partial_z \boldsymbol{u} = \boldsymbol{u} \cdot \nabla \boldsymbol{u}$
2. Diffusion + forcing: $\partial_t \boldsymbol{u} = \nu \Delta \boldsymbol{u} + \beta T_{\boldsymbol{e}_z}$
3. Projection step (Incompressibility): $\boldsymbol{u}^{k+1} = \boldsymbol{u}^* - \Delta \boldsymbol{p}, \quad \text{with } \nabla \cdot \boldsymbol{u}^{k+1} = 0$
   The pressure acts like a Lagrange multiplier, enforcing the incompressibility constraint, $\nabla \boldsymbol{u} = 0$.

To compute the numerical solution for the velocity field, the problem is solved in two stages.

1. **Step 1: Compute intermediate velocity**
   Ignoring the incompressibility constraint, we solve $\partial_t \boldsymbol{u} + \boldsymbol{u} \cdot \nabla \boldsymbol{u} = \nu \cdot \Delta \boldsymbol{u} + f$. This yields an intermediate velocity $\boldsymbol{u}^*$ that generally does not satisfy the divergence-free condition, $\nabla \cdot \boldsymbol{u}^* \neq 0$
2. **Step 2: Projection step**
   The intermediate velocity is then corrected to enforce incompressibility:

   $$\boldsymbol{u}^{k+1} = \boldsymbol{u}^* - \nabla \boldsymbol{p}.$$

   Using the *Helmholtz decomposition*,

   $$\boldsymbol{u}^* = \boldsymbol{u}^{\perp} + \nabla \phi,$$

   where

   $$\boldsymbol{u}^{\perp}$$

   is divergence free. Taking the divergence yields

   $$\nabla \cdot \boldsymbol{u}^* = \Delta \phi.$$

Imposing incompressibility on $\boldsymbol{u}^{k+1}$,

$$\nabla \cdot \boldsymbol{u}^{k+1} = 0,$$

leads to the Poisson equation for the pressure:

$$\Delta p = \nabla \cdot \boldsymbol{u}^{*}.$$

Here, we briefly summarize the developments so far. Readers who feel confident with the material presented above may skip the following paragraphs. Nevertheless, we provide a short overview of the underlying problem to make the discussion in the later sections easier to follow.

We begin by giving context to the physical problem addressed by the Navier–Stokes equations.

The Navier-Stokes equations are partial differential equations that are generally difficult to solve analytically. Consequently, numerical methods are necessary to compute the velocity on our domain.

The physical problem described by the Navier-Stokes can be interpreted as follows. Certain properties of the fluid must be satisfied, such as how the volume changes under pressure, how the flow evolves with time, how the motion at one point is influenced by surrounding fluid, etc. These properties are represented by different operators that describe the evolution of the flow. Our main interest, therefore, is the velocity of the fluid at each grid point in our domain at each timestep. The temporal development of the velocity at a given location is determined by spatial characteristics at that location, such as the first and second spatial derivatives of the velocity relative to the surrounding fluid. In other words, the *temporal* evolution is driven by *spatial* derivatives of the velocity itself. Advection, for example, describes the transport of fluid along the direction of flow. Diffusion, on the other hand, models the smoothing of velocity differences within the domain. Therefore, in the absence of any external force or energy, input would cause unevenly distributed velocity behavior, and we would observe an eventual state of rest. These mechanisms depend only on characteristics of the velocity field $\mathbf{u}$. However, additional terms such as buoyancy forcing βT_z and pressure gradient $\frac{1}{\rho}\nabla p$ introduce additional complexity. In the following sections, we neglect additional forcing terms (which could represent, for example, heating to induce convection) and focus primarily on the effects of the pressure term.

Consider a generalized incompressible fluid, characterized by $\nabla \cdot \mathbf{u} = 0$. By definition,

an incompressible fluid does not change its volume when forces or pressure are applied. Consequently, no volume should be lost or gained during the flow. Pressure, which is coupled to the velocity field, therefore becomes an additional dynamical variable that must be solved for.

If the governing equation only depends on the velocity field $\mathbf{u}$, we could simply discretize the equation and solve numerically for $\mathbf{u}$. But with the presence of the pressure term p, this makes the problem more complicated. To address this, we break the numerical process into two stages. First, the coupling between velocity $\mathbf{u}$ and pressure p is temporarily ignored and the incompressibility condition $\nabla \mathbf{u} = 0$ is not enforced. This step yields an intermediate velocity $\mathbf{u}^*$ for the velocity for the next timestep. Secondly, the pressure is introduced to enforce the incompressibility constraint. This can be done efficiently by applying the *Helmholtz decomposition* to the velocity field. As a result, the pressure can be determined by solving the Poisson equation, which ensures the corrected velocity field satisfies the divergence-free condition.

We have obtained numerical solutions for an intermediate velocity and pressure that satisfies the incompressibility constraint. Using these quantities, we can compute the velocity at the next discrete timestep while preserving incompressibility. A detailed description of this procedure is given in Chapter 2.2.

Operator splitting for the Velocity Equation

Returning to the operators of the velocity equation, we now apply operator splitting for all the different terms. We begin with the advection operator

$$\partial_t \boldsymbol{u} + a_y \partial_y \boldsymbol{u} + a_z \partial_z \boldsymbol{u} = 0$$

Here we define $\boldsymbol{a} = \begin{pmatrix} a_y(y,z) \\ a_z(y,z) \end{pmatrix}$ and $\boldsymbol{u} = \begin{pmatrix} v \\ w \end{pmatrix}$

The operator splitting is first applied in their spatial directions. This leads to two one-dimensional advection problems:

1. Solve

$$\partial_t \boldsymbol{u} + a_y \partial_y \boldsymbol{u} = 0$$

Or, more explicitly written component wise:

$$\begin{cases} \partial_t v + a_y \partial_y v & = 0 \\ \partial_t w + a_y \partial_y w & = 0 \end{cases}$$

We do the same for the second spatial dimension:

2. Solve

$$\partial_t \boldsymbol{u} + a_z \partial_z \boldsymbol{u} = 0$$

Again, more explicitly written component wise:

$$\begin{cases} \partial_t v + a_z \partial_z \boldsymbol{u} & = 0 \\ \partial_t w + a_z \partial_z w & = 0 \end{cases}$$

3. Apply Lie Splitting in time: $\boldsymbol{u}^{k+1} = A_z(\Delta t) A_y(\Delta t) \boldsymbol{u}^k$

We have now obtained an operator splitting for the (linear) advection equation. In the next section, we will examine the upward finite difference method to solve the resulting equations numerically.

2.1.4 Upwind Finite Difference Method

To introduce the upwind finite difference method, we consider the one-dimensional linear advection equation

$$\frac{\partial_u}{\partial_t} + a \frac{\partial u}{\partial x} = 0.$$

This equation describes a wave propagating along the x-axis with velocity a. If $a > 0$, the wave propagates to the right. Thus, an upwind direction would correspond to the direction in the left, towards smaller x values. Conversely, if $a < 0$, the wave propagates to the left, and the upwind direction corresponds to larger values of x.

Therefore, we will use backward finite difference methods for $a > 0$ and forward finite difference methods for $a < 0$. For the one-dimensional discretization we define

$$x_i := i\,\Delta x, \quad i = 0, \ldots, N$$
$$t^n := n\,\Delta t$$

also define discrete no of steps for Time? So maybe

1. $a > 0$ (backward):

$$\partial_x \boldsymbol{u} \approx \frac{\boldsymbol{u}_i - \boldsymbol{u}_{i-1}}{\Delta x}$$

2. $a < 0$ (forward):

$$\partial_x \boldsymbol{u} \approx \frac{\boldsymbol{u}_{i+1} - \boldsymbol{u}_i}{\Delta x}$$

Using a forward Euler discretization in time,

$$\frac{\boldsymbol{u}_{i+1}^n - \boldsymbol{u}_i^{n+1}}{\Delta t} = -a(\partial_x \boldsymbol{u})_i^n$$

1. $a > 0$ (backward):

$$\frac{\boldsymbol{u}_{i+1}^n - \boldsymbol{u}_i^{n+1}}{\Delta t} = -a(\partial_x \boldsymbol{u})_i^n \tag{2.15}$$

$$\Rightarrow \boldsymbol{u}_i^{n+1} - \boldsymbol{u}_i^n = -a\frac{\Delta t}{\Delta x}(\boldsymbol{u}_i^n - \boldsymbol{u}_{i-1}^n) \tag{2.16}$$

$$\Rightarrow \boldsymbol{u}_i^{n+1} = \left(1 - a\frac{\Delta t}{\Delta x}\right)\boldsymbol{u}_i^n + a\frac{\Delta t}{\Delta x}\boldsymbol{u}_{i-1}^n \tag{2.17}$$

$$\Rightarrow \boldsymbol{u}_i^{n+1} = A_1 \boldsymbol{u}_i^n \tag{2.18}$$

2. $a < 0$ (forward):

$$\frac{\boldsymbol{u}_{i+1}^n - \boldsymbol{u}_i^{n+1}}{\Delta t} = -a(\partial_x \boldsymbol{u})_i^n \tag{2.19}$$

$$\Rightarrow \boldsymbol{u}_i^{n+1} - \boldsymbol{u}_i^n = -a\frac{\Delta t}{\Delta x}(\boldsymbol{u}_{i+1}^n - \boldsymbol{u}_i^n) \tag{2.20}$$

$$\Rightarrow \boldsymbol{u}_i^{n+1} = \left(1 - a\frac{\Delta t}{\Delta x}\right)\boldsymbol{u}_i^n - a\frac{\Delta t}{\Delta x}\boldsymbol{u}_{i+1}^n \tag{2.21}$$

$$\Rightarrow \boldsymbol{u}_i^{n+1} = A_2 \boldsymbol{u}_i^n \tag{2.22}$$

Thus, we obtain an operator for each upwind case, depending only on the sign of a. To compactly represent the update for all grid points, we write the scheme in matrix form.

$$A_1 = \begin{pmatrix} \left(1 - a\frac{\Delta t}{\Delta x}\right) & 0 & & \dots & 0 \\ a\frac{\Delta t}{\Delta x} & \ddots & & & \vdots \\ 0 & \ddots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & 0 \\ 0 & \dots & 0 & a\frac{\Delta t}{\Delta x} & \left(1 - a\frac{\Delta t}{\Delta x}\right) \end{pmatrix}$$

$$A_2 = \begin{pmatrix} \left(1 - a\frac{\Delta t}{\Delta x}\right) & -a\frac{\Delta t}{\Delta x} & 0 & \dots & 0 \\ 0 & \ddots & \ddots & & \vdots \\ & & \ddots & \ddots & 0 \\ \vdots & & & \ddots & -a\frac{\Delta t}{\Delta x} \\ 0 & \dots & & 0 & \left(1 - a\frac{\Delta t}{\Delta x}\right) \end{pmatrix}$$

Stability of the Operators

In general, a matrix operator A is stable if its infinity norm satisfies $\|A\|_\infty \leq 1$. We investigate both operators A_1 (corresponding to the case $a > 0$ and therefore backward finite difference method) and A_2 (corresponding to the case $a < 0$ and therefore forward finite difference method).

$$\|A_1\|_\infty = \left|a\frac{\Delta t}{\Delta x}\right| + \left|1 - a\frac{\Delta t}{\Delta x}\right| \leq 1 \tag{2.23}$$

$$\Rightarrow^{a>0} \left|1 - a\frac{\Delta t}{\Delta x}\right| \leq 1 - a\frac{\Delta t}{\Delta x} \tag{2.24}$$

$$\Rightarrow 1 - a\frac{\Delta t}{\Delta x} \geq 0 \tag{2.25}$$

$$\Rightarrow a\frac{\Delta t}{\Delta x} \leq 1 \tag{2.26}$$

Repeating this argument for our second operator A_2 yields

$$\|A_2\|_\infty = \left|a\frac{\Delta t}{\Delta x}\right| + \left|1 - a\frac{\Delta t}{\Delta x}\right| \leq 1 \tag{2.27}$$

$$\Rightarrow^{a>0} \left|1 - a\frac{\Delta t}{\Delta x}\right| \leq 1 + a\frac{\Delta t}{\Delta x} \tag{2.28}$$

$$\Rightarrow 1 + a\frac{\Delta t}{\Delta x} \geq 0 \tag{2.29}$$

$$\Rightarrow -a\frac{\Delta t}{\Delta x} \leq 1 \tag{2.30}$$

Combining both cases yields the general stability condition

$$\left|a\frac{\Delta t}{\Delta x}\right| \leq 1,$$

which is the CFL condition for the upwind scheme.

2.1.5 Two-Dimensional Upwind Scheme with Operator Splitting

Now we jointly apply operator splitting with the upwind finite difference method for the two-dimensional advection equation. We define $u_{ij}^n := u(t^n, y_i, z_i)$ and assume an equidistant spatial grid. The spacings Δy and Δz denote the distances between neighboring grid points.

We first solve the advection term in the y direction. For a fixed value z_j, the equation becomes

$$\partial_t \boldsymbol{u} + a_y \partial_y \boldsymbol{u} = 0.$$

The spatial derivative is approximated using an upwind scheme, depending on the sign of a_y:

$$(\partial_y \boldsymbol{u})_{ij} = \begin{cases} \frac{\boldsymbol{u}_{ij} - \boldsymbol{u}_{i-1\,j}}{\Delta y} & a_y > 0 \\ \frac{\boldsymbol{u}_{i+1\,j} - \boldsymbol{u}_{i\,j}}{\Delta y} & a_y < 0 \end{cases}$$

Using a forward Euler step in time yields

$$u_{ij}^* = u_{ij}^n - \Delta t a_{y,ij} (\partial_y \boldsymbol{u})_i j.$$

Stability requires the CFL condition

$$a_y \frac{\Delta t}{\Delta y} \leq 1.$$

We use this result to proceed with the solution of the z-coordinate corresponding to the second operator:

$$\partial_t \boldsymbol{u} + a_z \partial_z \boldsymbol{u} = 0$$

We define analogously

$$(\partial_y \boldsymbol{u})_{ij} = \begin{cases} \frac{\boldsymbol{u}_{ij} - \boldsymbol{u}_{i-1\,j}}{\Delta y} & a_y > 0 \\ \frac{\boldsymbol{u}_{i+1\,j} - \boldsymbol{u}_{i\,j}}{\Delta y} & a_y < 0 \end{cases}$$

Using the result from the first split operator, we now compute

$$u_{ij}^{n+1} = u_{ij}^{*} - \Delta t a_{z,ij} (\partial_z \boldsymbol{u})_i j$$

We ensure stability in this case as well by respecting $\frac{|a_z|\Delta t}{\Delta z} \leq 1$.

We can now derive a stability condition for all of our discretized points in space. We first obtain

$$\max |a_y| = \max_{i,j} |a_y(y_i, z_j)|$$

and

$$\max |a_z| = \max_{i,j} |a_z(y_i, z_j)|.$$

With the help of these variables, we deduce the overall constraint for Δt, that is necessary to ensure stability:

$$\Delta t \leq \min \left(\frac{\Delta y}{\max |a_y|}, \frac{\Delta z}{\max |a_z|} \right)$$

In summary, we have implemented operator splitting together with the upwind finite difference method for the two-dimensional linear advection equation and derived a stability condition that must be satisfied over the entire computational domain.

boundary conditions implementation

2.2 Incompressible Flow

2.2.1 Problem Setup

We consider the incompressible Navier–Stokes equations for velocity field $\mathbf{u} = (v, w)$ and pressure p,

$$\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla)\mathbf{u} + \frac{1}{\rho}\nabla p = \nu \, \Delta \mathbf{u}, \tag{2.31}$$

$$\nabla \cdot \mathbf{u} = 0, \tag{2.32}$$

where ρ is the fluid density and ν the kinematic viscosity. The buoyancy parameter β is set to zero throughout this section, so that (2.31) describes a purely viscous incompressible flow without external forcing.

The computational domain is the two-dimensional rectangle $\Omega = [0, L_y) \times [0, L_z]$. The y direction is periodic and the z direction is wall-bounded, reflecting a channel-like geometry.

Periodicity in y. The velocity field satisfies

$$\mathbf{u}(t, 0, z) = \mathbf{u}(t, L_y, z) \qquad \forall\, t \geq 0,\ z \in [0, L_z], \tag{2.33}$$

so that the domain wraps around in y and no boundary treatment is required in that direction.

Wall conditions at $z = 0$ and $z = L_z$. Two conditions are imposed on each solid wall. First, no fluid may penetrate the boundary (*no-penetration*):

$$w(t, y, 0) = w(t, y, L_z) = 0 \qquad \forall\, t \geq 0,\ y \in [0, L_y). \tag{2.34}$$

Second, the tangential component v is subject to a *free-slip* condition,

$$\left.\frac{\partial v}{\partial z}\right|_{z=0,\, L_z} = 0, \tag{2.35}$$

allowing the flow to move freely parallel to the wall without friction while exerting no viscous shear stress on it. Numerically, (2.35) is enforced via a ghost-cell extrapolation: the value of v at each wall is node is set equal to the value at the adjacent interior node.

2.2.2 Initial Conditions

For projection methods, it is advantageous to begin with an initial velocity field that is analytically divergence-free. If the initial condition contains a large divergence, the first projection step must remove it, which can introduce sizable error into the velocity and pressure fields. Therefore, to avoid this issue, we initialize the flow with velocity fields that satisfy $\nabla \cdot \mathbf{u}_0 = 0$. The built-in modes take the form

$$v_0(y,z) = \sin(n_y k_y y)\cos(n_z k_z z), \tag{2.36}$$

$$w_0(y,z) = -\frac{n_y k_y}{n_z k_z}\cos(n_y k_y y)\sin(n_z k_z z), \tag{2.37}$$

where $k_y = 2\pi/L_y$ and $k_z = \pi/L_z$ are the fundamental wavenumbers in each direction, and $n_y, n_z \in \mathbb{Z}^+$ select the mode numbers.

Divergence-free condition. Taking the divergence of (2.36)–(2.37) gives

$$\partial_y v_0 + \partial_z w_0 \tag{2.38}$$

$$= n_y k_y \cos(n_y k_y y)\cos(n_z k_z z) - n_y k_y \cos(n_y k_y y)\cos(n_z k_z z) = 0, \tag{2.39}$$

so $\nabla \cdot \mathbf{u}_0 = 0$ holds pointwise throughout the domain.

Boundary conditions. The choice of $k_z = \pi/L_z$ places a node of $\sin(n_z k_z z)$ at both $z = 0$ and $z = L_z$ for every $n_z \in \mathbb{Z}^+$ so

$$w_0(y,0) = w_0(y,L_z) = 0 \quad \forall\, y, \tag{2.40}$$

and the no-penetration wall condition is satisfied by construction. The periodicity in the y direction is automatically satisfied by the sinusoidal dependence.

Implemented modes. The implemented modes are summarized in Table 2.1. The `double` mode is the linear superposition

$$v_0 = \sin(k_y y)\cos(k_z z) + \sin(2k_y y)\cos(k_z z), \tag{2.41}$$

$$w_0 = -\frac{k_y}{k_z}\cos(k_y y)\sin(k_z z) - \frac{2k_y}{k_z}\cos(2k_y y)\sin(k_z z), \tag{2.42}$$

which is divergence-free by linearity. The solver also accepts a `custom` mode in which the user supplies arbitrary functions $v_0(y, z)$ and $w_0(y, z)$ directly. In this case, the user is responsible for ensuring that the supplied field satisfies $\nabla \cdot \mathbf{u}_0 = 0$ and the wall boundary conditions.

Mode	n_y	n_z
`mode1`	1	1
`mode2`	2	1
`mode3`	1	2
`double`	superposition of `mode1` and `mode2`	

Table 2.1: **Initial condition modes.** Each built-in mode is characterized by its y- and z-wavenumber multipliers (n_y, n_z). The `double` mode is the linear superposition of `mode1` and `mode2`; the `custom` mode accepts user-supplied velocity functions.

2.2.3 Time Step Selection

Because the predictor step uses an explicit time discretization for both advection and diffusion, the time step Δt must satisfy stability conditions imposed by each operator independently. The overall time step is chosen as

$$\Delta t = \alpha \min(\Delta t_{\text{adv}},\, \Delta t_{\text{diff}})\,, \tag{2.43}$$

where $\alpha \in (0, 1)$ is a safety factor (set to $\alpha = 0.4$ in our implementation) and Δt_{adv}, Δt_{diff} are the stability limits from advection and diffusion, respectively.

Advection limit (CFL condition). For first-order upwind advection on a uniform grid, the Courant–Friedrichs–Lewy (CFL) condition requires

$$\frac{\max|\mathbf{u}|\,\Delta t}{\Delta x} \leq 1 \tag{2.44}$$

in each spatial direction. Applying this to both the y- and z-directions gives the advective time step limit

$$\Delta t_{\text{adv}} = \min\left(\frac{\Delta y}{\max|v|},\, \frac{\Delta z}{\max|w|}\right), \tag{2.45}$$

where $\max|v|$ and $\max|w|$ denote the global maxima of the velocity magnitudes over the entire domain at the current time level. A small floor value is applied to each velocity

maximum before division to prevent division by zero in the case of a quiescent initial field.

Diffusion limit (von Neumann condition). For the explicit discretization of the diffusion term $\nu\Delta\mathbf{u}$, stability requires that the diffusive CFL number remain bounded. On a two-dimensional uniform grid this yields the von Neumann stability condition

$$\Delta t_{\text{diff}} \leq \frac{1}{2\nu}\left(\frac{1}{\Delta y^2}+\frac{1}{\Delta z^2}\right)^{-1}, \tag{2.46}$$

which is independent of the velocity field and depends only on the kinematic viscosity ν and the grid spacings.

Combined condition. The time step (2.43) takes the minimum of both limits and scales it by the safety factor α, ensuring a margin against the onset of instability as the velocity field evolves. At each time step, the CFL numbers

$$C_y = \frac{\max|v|\,\Delta t}{\Delta y}, \qquad C_z = \frac{\max|w|\,\Delta t}{\Delta z} \tag{2.47}$$

are monitored, and a warning is issued if either exceeds 1. Note that Δt is computed once from the initial velocity field and held fixed throughout the simulation; it is not recomputed adaptively at each step.

2.2.4 Chorin's Projection Method

We consider again the incompressible Navier–Stokes equations

$$\partial_t\mathbf{u} + \mathbf{u}\cdot\nabla\mathbf{u} + \frac{1}{\rho}\nabla p = \nu\Delta\mathbf{u} \qquad (\beta = 0), \tag{2.48}$$

$$\nabla\cdot\mathbf{u} = 0, \tag{2.49}$$

on the domain $\Omega = [0, L_y]\times[0, L_z]$, subject to periodic boundary conditions in y and no-penetration / free-slip conditions at the walls $z = 0$ and $z = L_z$. To solve this system, we employ Chorin's *projection method*, which decouples the velocity update from the pressure solve by splitting the computation into three steps at each timestep:

1. **Predictor Step** — advance the momentum equation without the pressure gradient to obtain an intermediate velocity $\mathbf{u}^*$, which is generally not divergence-free.

2. **Pressure Poisson Equation** — compute the pressure correction p by solving a Poisson equation derived from the incompressibility constraint.

3. **Projection Step** — correct $\mathbf{u}^*$ using ∇p to recover a divergence-free velocity $\mathbf{u}^{n+1}$.

This procedure is a form of operator splitting. The nonlinear advection, diffusion, and incompressibility constraint are each treated in a dedicated sub-step rather than simultaneously.

Predictor Step

The predictor step advances the momentum equation over one time step Δt, while temporarily ignoring the pressure gradient. The intermediate velocity $\mathbf{u}^* = (v^*, w^*)$ is obtained from the explicit Euler discretization

$$\mathbf{u}^* = \mathbf{u}^n + \Delta t\Big[-(\mathbf{u}^n \cdot \nabla)\mathbf{u}^n + \nu\,\Delta \mathbf{u}^n\Big], \tag{2.50}$$

or written componentwise,

$$v^* = v^n - \Delta t\left(v^n \frac{\partial v^n}{\partial y} + w^n \frac{\partial v^n}{\partial z}\right) + \Delta t\,\nu\,\Delta v^n, \tag{2.51}$$

$$w^* = w^n - \Delta t\left(v^n \frac{\partial w^n}{\partial y} + w^n \frac{\partial w^n}{\partial z}\right) + \Delta t\,\nu\,\Delta w^n. \tag{2.52}$$

The right-hand side of (2.50) is split into an advection contribution and a diffusion contribution, each treated separately.

Advection. The advection terms in (2.51)–(2.52) are discretized using the operator-split first-order upwind scheme described in Section 2.1.5. The two velocity components v and w are each advected by the full velocity field $\mathbf{u}^n = (v^n, w^n)$, with the y-sweep applied first, followed by the z-sweep:

$$v_{\text{adv}} = A_z(\Delta t)\,A_y(\Delta t)\,v^n, \tag{2.53}$$

$$w_{\text{adv}} = A_z(\Delta t)\,A_y(\Delta t)\,w^n, \tag{2.54}$$

where A_y and A_z denote the one-dimensional upwind operators in the y- and z-directions respectively.

Diffusion. The diffusion increment is evaluated on the *original* velocity field $\mathbf{u}^n$, not on the intermediate advected field $\mathbf{u}_{\text{adv}}$. This is consistent with an explicit Euler splitting in which advection and diffusion are treated as independent operators acting on the same base state. The intermediate velocity is then assembled as

$$v^* = v_{\text{adv}} + \Delta t\, \nu\, \Delta v^n, \tag{2.55}$$

$$w^* = w_{\text{adv}} + \Delta t\, \nu\, \Delta w^n, \tag{2.56}$$

where Δ denotes the discrete Laplacian.

Boundary conditions. Wall boundary conditions are enforced at two points during the predictor step. They are first applied to $\mathbf{u}^n$ before any stencil evaluation, ensuring that the finite-difference operators see a consistent velocity field at the boundaries. They are then re-applied to $\mathbf{u}^*$ after the advection and diffusion increments have been added, correcting any boundary values that may have been corrupted by the interior stencils. Specifically, the no-penetration condition $w^* = 0$ and free-slip condition $\partial_z v^* = 0$ are enforced at $z = 0$ and $z = L_z$.

Pressure Poisson Equation

Given the intermediate velocity $\mathbf{u}^*$ from the predictor step, the right-hand side of the Poisson equation is assembled as

$$f = \frac{\rho}{\Delta t} \nabla \cdot \mathbf{u}^*, \tag{2.57}$$

The pressure field p is then obtained by solving

$$\Delta p = f \quad \text{in } \Omega, \tag{2.58}$$

subject to Neumann conditions $\partial p / \partial \mathbf{n} = 0$ at the walls $z = 0$ and $z = L_z$, periodic conditions in y, and the zero-mean gauge $\langle p \rangle = 0$.

Compatibility Condition. The Neumann problem (2.58) admits a solution only if the right-hand side satisfies the compatibility condition

$$\int_\Omega f \, \mathrm{d}\Omega = 0. \tag{2.59}$$

This is enforced numerically by subtracting the spatial mean of f before solving, replacing f with $f - \langle f \rangle$.

Spectral Diagonalization. The Poisson equation is solved using a spectral direct solver that exploits the structure of the boundary conditions in each direction. In the periodic y-direction, the discrete operator is diagonalized by the Fast Fourier Transform (FFT). In the wall-bounded z-direction, the Neumann boundary conditions are compatible with the Discrete Cosine Transform of type I (DCT-I), which diagonalizes the corresponding Neumann Laplacian stencil. Applying both transforms to (2.58) yields a decoupled system of algebraic equations in spectral space,

$$(\lambda_k^y + \lambda_m^z)\,\hat{P}_{k,m} = \hat{F}_{k,m}, \tag{2.60}$$

where $\hat{P}_{k,m}$ and $\hat{F}_{k,m}$ are the spectral coefficients of p and f respectively, and the eigenvalues of the discrete Laplacian are

$$\lambda_k^y = \frac{2}{\Delta y^2}\left(\cos\frac{2\pi k}{N_y} - 1\right), \quad k = 0, \ldots, N_y - 1, \tag{2.61}$$

$$\lambda_m^z = \frac{2}{\Delta z^2}\left(\cos\frac{\pi m}{N_z - 1} - 1\right), \quad m = 0, \ldots, N_z - 1. \tag{2.62}$$

These eigenvalues are the exact eigenvalues of the second-order central-difference operators in each direction: (2.61) corresponds to the periodic tridiagonal stencil on N_y points, and (2.62) corresponds to the Neumann-reflected tridiagonal stencil on N_z points with endpoints included.

Gauge Fix and Inversion. The $(k, m) = (0, 0)$ mode corresponds to $\lambda_0^y + \lambda_0^z = 0$, so (2.60) is singular for this mode. Physically, this reflects the fact that the Neumann problem determines p only up to an additive constant. The zero mode is handled by setting $\hat{P}_{0,0} = 0$, which enforces the zero-mean gauge $\langle p \rangle = 0$. All other modes are inverted as

$$\hat{P}_{k,m} = \frac{\hat{F}_{k,m}}{\lambda_k^y + \lambda_m^z}, \quad (k, m) \neq (0, 0). \tag{2.63}$$

The pressure field is then recovered by applying the inverse DCT-I in z followed by the inverse FFT in y. A final subtraction of the spatial mean is applied to enforce $\langle p \rangle = 0$ to floating-point precision.

Treatment of complex transforms. Because the right-hand side f is real-valued, the FFT in y produces complex coefficients. Since the scipy DCT implementation operates on real arrays only, the DCT-I is applied separately to the real and imaginary parts of the FFT output, and the results are recombined before inversion. The imaginary part of the final result is discarded, as it arises only from floating-point rounding.

Projection Step

Given the pressure field p obtained from solving the Poisson equation, the intermediate velocity $\mathbf{u}^*$ is corrected by removing its irrotational component. The corrected velocity at the new time level is

$$\mathbf{u}^{n+1} = \mathbf{u}^* - \frac{\Delta t}{\rho}\nabla p. \tag{2.64}$$

Evaluating (2.64) componentwise gives

$$v^{n+1} = v^* - \frac{\Delta t}{\rho}\frac{\partial p}{\partial y}, \tag{2.65}$$

$$w^{n+1} = w^* - \frac{\Delta t}{\rho}\frac{\partial p}{\partial z}. \tag{2.66}$$

Both partial derivatives of p are computed using the central-difference gradient operator, with Neumann ghost cells ($p_{i,0} = p_{i,1}$, $p_{i,N_z-1} = p_{i,N_z-2}$) applied at the walls to enforce $\partial p/\partial z = 0$ there, and periodic differences via `np.roll` in the y-direction.

To verify that (2.65)–(2.66) enforces incompressibility, take the divergence of (2.64):

$$\nabla \cdot \mathbf{u}^{n+1} = \nabla \cdot \mathbf{u}^* - \frac{\Delta t}{\rho}\Delta p. \tag{2.67}$$

Substituting the Poisson equation, $\Delta p = (\rho/\Delta t)\,\nabla \cdot \mathbf{u}^*$, yields

$$\nabla \cdot \mathbf{u}^{n+1} = \nabla \cdot \mathbf{u}^* - \nabla \cdot \mathbf{u}^* = 0, \tag{2.68}$$

so $\mathbf{u}^{n+1}$ is divergence-free to within the accuracy of the Poisson solver. In practice, the residual divergence $\|\nabla \cdot \mathbf{u}^{n+1}\|_\infty$ is monitored at each step as a convergence diagnostic.

2.2.5 Summary

The complete numerical procedure at each time step $t^n \to t^{n+1}$ is as follows:

1. **Predictor.** Solve
$$\mathbf{u}^* = \mathbf{u}^n + \Delta t\Big[-(\mathbf{u}^n \cdot \nabla)\mathbf{u}^n + \nu\Delta\mathbf{u}^n\Big],$$
using operator-split upwind advection followed by explicit diffusion. Apply wall boundary conditions to $\mathbf{u}^*$.

2. **Poisson solve.** Compute $\nabla \cdot \mathbf{u}^*$ and solve
$$\Delta p = \frac{\rho}{\Delta t}\nabla \cdot \mathbf{u}^*$$
via the spectral direct solver, enforcing zero mean pressure.

3. **Projection.** Correct the velocity,
$$\mathbf{u}^{n+1} = \mathbf{u}^* - \frac{\Delta t}{\rho}\nabla p,$$
and apply wall boundary conditions to $\mathbf{u}^{n+1}$.

2.3 Rayleigh-Bernard convection

So far we have ignored any forcing in the velocity equation of the Navier-Stokes equation. In the following section we want to introduce thermal forcing to reproduce the Rayleigh-Bernard convection cells. Just to name one example, they occur in climatology on different scales and convective phenomena are important in various proccesses all over the global atmospheric system. In this section we want to find adequate initial conditions and parameters to reproduce such a convective behaviour using our solutions.

Physical parameters

To start, we once again want to consider the momentum equation. For our problem, we will use the Boussinesq-approximation

.

explain Boussinesq approxima

Boussinesq Approximation

Accourding to the Boussinesq approximation we can simplify the flows driven by temperature differences (natural convection). It indicates that density changes are ignored in most parts of the equations such as inertia terms, etc. However, density changes are only considered

in the gravity term to calculate the buoyancy term. We are allowed to consider such an assumption if the temperature differences in the fluid are relatively small, The variation in density is much smaller than the mean density; and when the flow velocity is low, with the Mach number less than 0.3. In other circumstances, we are not permitted to assume the Boussinesq approximation.

This leads to

$$\partial_t \mathbf{u} + (\mathbf{u}) \cdot \nabla + \frac{1}{\rho} \nabla p = \nu \nabla \mathbf{u} + T \beta e_z$$

As we are interested in a certain set of parameters, we briefly want to give some inuitive idea of what the parameters are describing. Consider ν for example: If ν is too high (modelling too much friction), the fluid will remain in a purely conductive state, implying the absense of convective cells. The buyoancy parameter β is of importance, too. If β is too low, then heat will simply diffuse across the fluid instead of creating motion due to located heat forcing. We see, that these parameters are essential for a successful recreation of the convective cells. While viscosity ν determines, in how far the fluid is 'allowed' to react to the thermal forcing *beta* represents the strength of the thermal coupling.

In a dimensionless, more general context, the onset of convection is determined by the Rayleigh number. In our case, the Rayleigh number is directly proportional to $\frac{\beta}{\nu}$. We have to choose β large enough (or ν small) to overcome the internal friction and thermal diffusion of the fluid.

2.3.1 Initial conditions

To trigger the desired convection cells we not only are in need of an accurate parameterization. We also rely on appropiate initial conditions. As symmetry in the initial conditions could hinder the formation of convective cells, we can not start with a perfectly uniform or stratified fluid.

Instead, we are suggesting the following characteristics for the initial conditions in our context. Firstly, we want to implement a conductive profile. That means, that we start with a temperature field $T_0(y, z)$ that roughly follows the linear profile $1 - z$ (hot at the bottom, cold at the top).

Secondly, we want to break the symmetry in the velocity field by the introduction of small perturbations. We chose random noise that is used to initialize the velocity field $\mathbf{u}_0(y, z)$. These perturbations will be amplified by the the buoyancy term $T\beta e_z$ and, if the Rayleigh number is high enough, eventually lead to the emergence of Rayleigh-Bernard convection.

Formally, the described set of initial conditions can be described in the following way. On

our domain $\Omega = [0,1]^2$ we introduce time-invariant temperatures (constant heating across time): In a perfectly still fluid ($\mathbf{u} = 0$) the temperature settles into a linear conductivity profile.

$$T(x, y) = 1 - z$$

This set up implies $T = 1$ at $z = 0$ (bottom) and $T = 0$ at $z = 1$ (top). Note, that this state satisfies the equilibrium condition meaning

$$\partial_t T + \mathbf{u}\Delta T = \Delta T$$

where $\Delta T = 0$ and $\mathbf{u} \cdot \Delta t = 0$.

To introduce the mentioned perturbations is the temperature field, we will add noise to these initial conditions.

$$T = (1 - z) + \epsilon$$

To understand, if such perturbations will grow or vanish, we substitute the temperature field into the momentum equations and linearize them. This is performed by dropping nonlinear terms like $\nu \cdot \nabla\epsilon$ or $(\mathbf{u} \cdot \nabla)\mathbf{u}$

That is how we obtain the temperature equations

$$\partial_t + u \cdot \nabla(1 - z) = \Delta\epsilon \tag{2.69}$$

$$\nabla(1 - z) = -e_z \tag{2.70}$$

$$\tag{2.71}$$

This leads to $\mathbf{u} \cdot (1 - z) = -w$ So in total we obtain the linearized heat equation

$$\partial_t \epsilon = w + \Delta\epsilon$$

We can see, that a vertical upward motion $w > 0$ acts as the source term that increases the local temperature perturbation.

If we ignore non-linear terms in the momentum equation and split decompose pressure into a static part and perturbation p', the vertical component of the momentum becomes:

$$\partial_t w + \frac{1}{\rho}\partial_z p' = \nu\Delta w + \epsilon\beta$$

Here, $\epsilon\beta$ is the buoyancy force. It drives the vertical velocity w, which in turn increases the perturbation ϵ. Thus, stability is is depending on buoyancy and damping.

The perturbations depend on two major forces:

1. **Driving forces**: The buoyancy term ($T\beta e_z$) which amplifies the disturbances as hot fluid is rising.

2. **Damping forces**: The viscosity $\nu\Delta\mathbf{u}$ and thermal diffusion ΔT, which act to 'smooth' velocity and temperature gradients.

In conclusion, we see that the growth rate of the perturbations ϵ become positive only when the buoyancy term overcomes these damping effects. If β is too small, diffusion is more important, and the perturbation decays back to the conductive state.

2.3.2 Solution Strategy

The simulation utilizes an Explicit Euler scheme for temporal advancement. While explicit methods are straightforward to implement, they impose a strict constraint on the time step, defined by the Courant-Friedrichs-Lewy (CFL) condition. In our implementation, we selected a grid to ensure the Courant number remains below 1. This prevents physical information from propagating faster than the numerical scheme, avoiding numerical divergence.

Requies intro paragraph

3 Results and Discussion

3.0.1 Numerical Verification of Advection

The first task involved testing the Upwind finite difference scheme using a $2D$ advection problem. The numerical results for the velocity components v and w were compared against the analytical solutions. For a 100×100 grid, we observed a maximum error of approximately 0.38 for v and 0.42 for w. This level of error is consistent with first-order numerical schemes, which are known for introducing numerical diffusion, causing the profiles to dampen over time.

3.0.2 Analysis of the Projection Method Step

The reduction of divergence from 1.82 to 0.05 demonstrates the sensitivity of the projection method to boundary conditions. The remaining divergence error (5.55%) is primarily attributed to the Jacobi iterative solver used for the Poisson equation. Since the Jacobi method has a **slow convergence rate for high-frequency errors**, increasing the number of iterations or employing **a more robust solver** like the Conjugate Gradient method (CG) or Fast Fourier Transform (FFT) would further drive the divergence towards machine epsilon (10^{-10} or smaller).

3.0.3 Stability and Time-Stepping

The use of an Explicit Euler scheme necessitates a strict adherence to the Courant (CFL) condition. In our simulations, a time step of $\Delta t = 0.001$ was chosen to maintain **a Courant number below 1**. Although an implicit scheme would allow for **larger time steps**, the current explicit implementation provides a clear and computationally straightforward approach to studying transient Rayleigh-Benard convection cells without the excessive computational costs of massive matrix inversions.

We considered two options for the numerical approach: Explicit Euler and Implicit Euler. If we chose Explicit, we had to tune the solver with many simple, "cheap" time steps governed by strict stability limits ($CFL < 1$). If we opted for the Implicit approach, we would have executed fewer, "expensive" time steps that are stable at larger intervals but

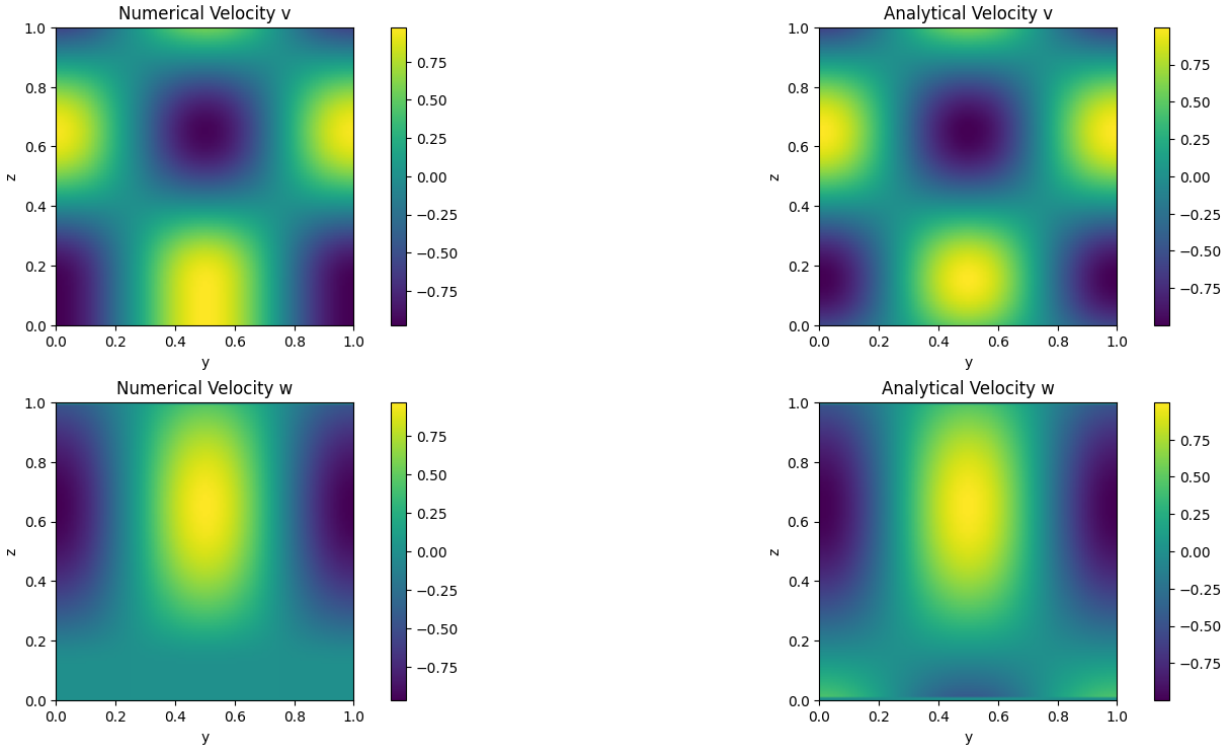


Figure 3.1: Comparison of analytical and numerical solutions in advection test.

require heavy matrix calculations. Eventually, due to fewer complexities and need for stronger processors, we employed the Explicit Euler method for our numerical approach.

3.0.4 Incompressible Flow Performance

The core of the simulation, the Chorin's projection method, was tested for mass conservation. In the initial implementation, the maximum divergence of the velocity field was high (1.82), indicating poor pressure correction. After refining the Neumann boundary conditions for the pressure Poisson solver and ensuring the grid spacing correctly accounts for the wall boundaries, the maximum divergence was significantly reduced to 5.55×10^{-2}. This confirms that the projection step is effectively enforcing the incompressibility constraint $\nabla \cdot \mathbf{u} \approx 0$.

3.0.5 Implementation Analysis

In this stage, the model was completed by incorporating the temperature (T) advection-diffusion equation and coupling it with the Navier-Stokes equations. The buoyancy term, scaled by coefficient β, was added to the vertical velocity (w) equation to simulate the thermal effects on fluid motion, which would be rising of warm air and sinking of cold air.

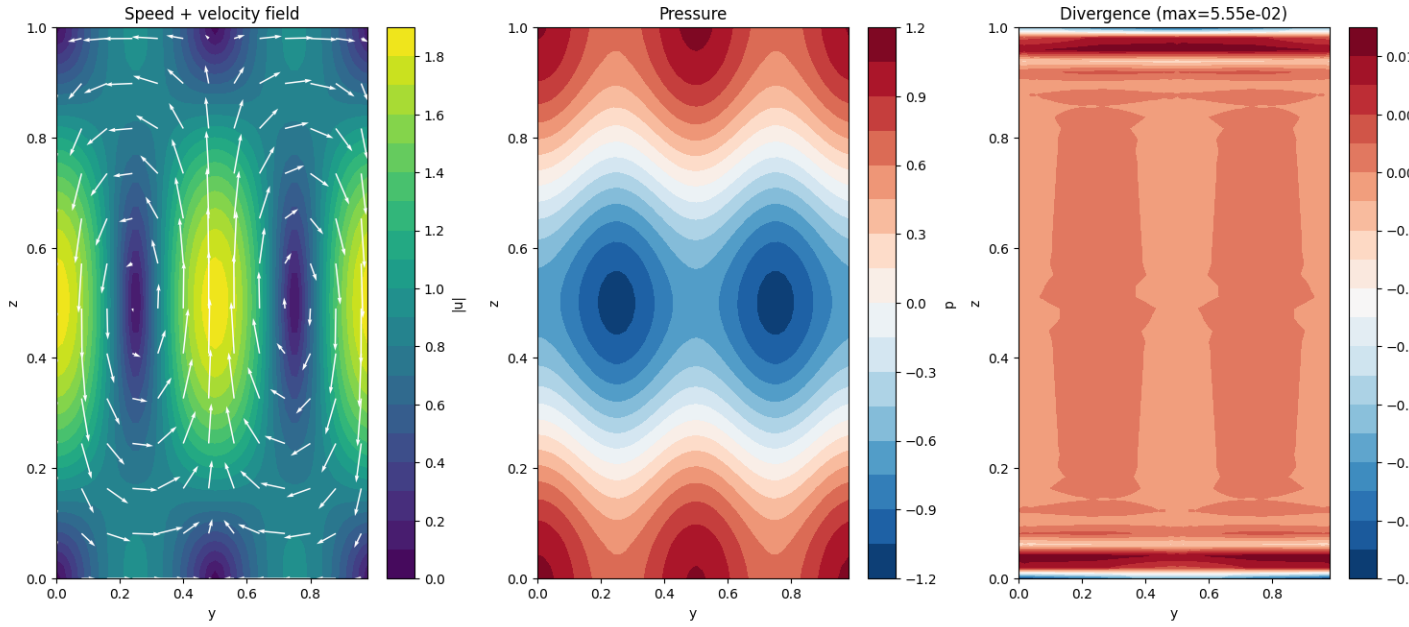


Figure 3.2: Initial Flow Field.

3.0.6 Poisson Solvers: Jacobi and FFT

The core of **Chorin's Projection method** is solving the Poisson equation for pressure. We explored two distinct approaches with vastly different outcomes.

Jacobi Iterative Method

Initially, the Jacobi method was used. It starts with an initial pressure guess and iteratively reduces the residual. However, as shown in the table below, this method is too slow for high-precision requirements. The initial divergence of 1.8 indicated that the pressure field was not correctly enforcing the incompressibility constraint.

Fast Fourier Transform (FFT) Method

In the final version, the Poisson solver was replaced with an FFT-based approach. This method transforms the problem from the spatial domain to the frequency domain, where complex derivatives become simple algebraic divisions.

3.0.7 Analysis of Divergence

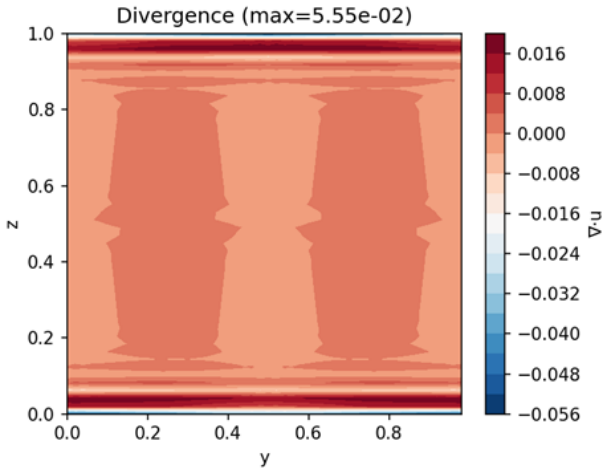


Figure 3.3: Divergence field in the Jacobi method. Max Divergence $= 5.55 \times 10^{-2}$.

3.0.8 FFT – Solving Divergence with FFT

The primary challenge in previous versions was high divergence caused by the limitations of the Jacobi iterative solver in handling the Poisson equation. By replacing it with the **Fast Fourier Transform (FFT)** method, the maximum divergence was reduced from 10^{-2} to approximately 10^{-15}. This transition not only achieved machine-precision accuracy but also significantly accelerated computational performance.

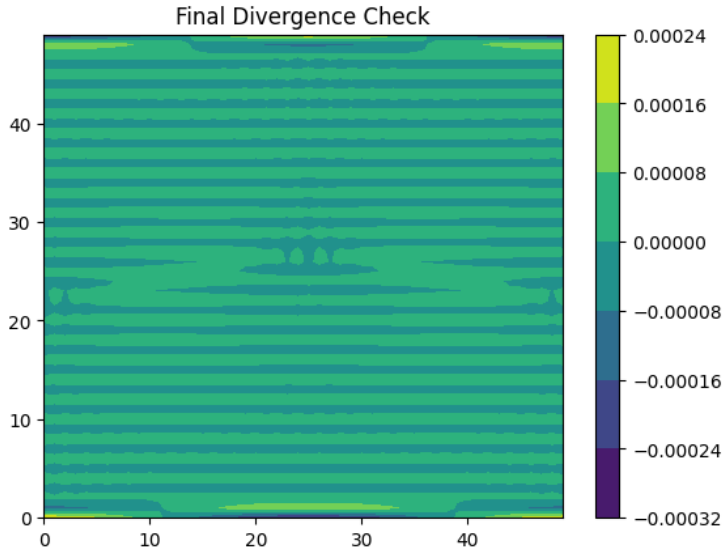


Figure 3.4: Achieving divergence error at machine precision accuracy using FFT solver.

Table 3.1: Table 1. Divergence Comparison

Solver Type	**Iterations / Method**	**Max Divergence**	**Status Analysis**
Initial Jacobi	1,000 Iterations	1.894	Unstable / Unacceptable
Optimized Jacobi	5,000 Iterations	5.55×10^{-2}	Moderate Accuracy (5% error)
FFT (Final Version)	Direct Solver	5.55×10^{-16}	Ideal (Machine Precision)

Reaching divergence $\approx 10^{-16}$ signifies that the mass conservation principle is perfectly satisfied. This resolves a major bottleneck in CFD codes, providing a robust foundation for the thermal simulation.

3.0.9 Rayleigh-Bénard Convection Results

Results from the updated code demonstrate the formation of classical convection cells. Warm plumes (red) rise from the bottom while cold plumes (blue) sink from the top. These circulation patterns confirm the simulation's success in capturing Rayleigh-Bénard physics and the stability of the numerical method under periodic (in y) and wall-bounded (in z) boundary conditions.

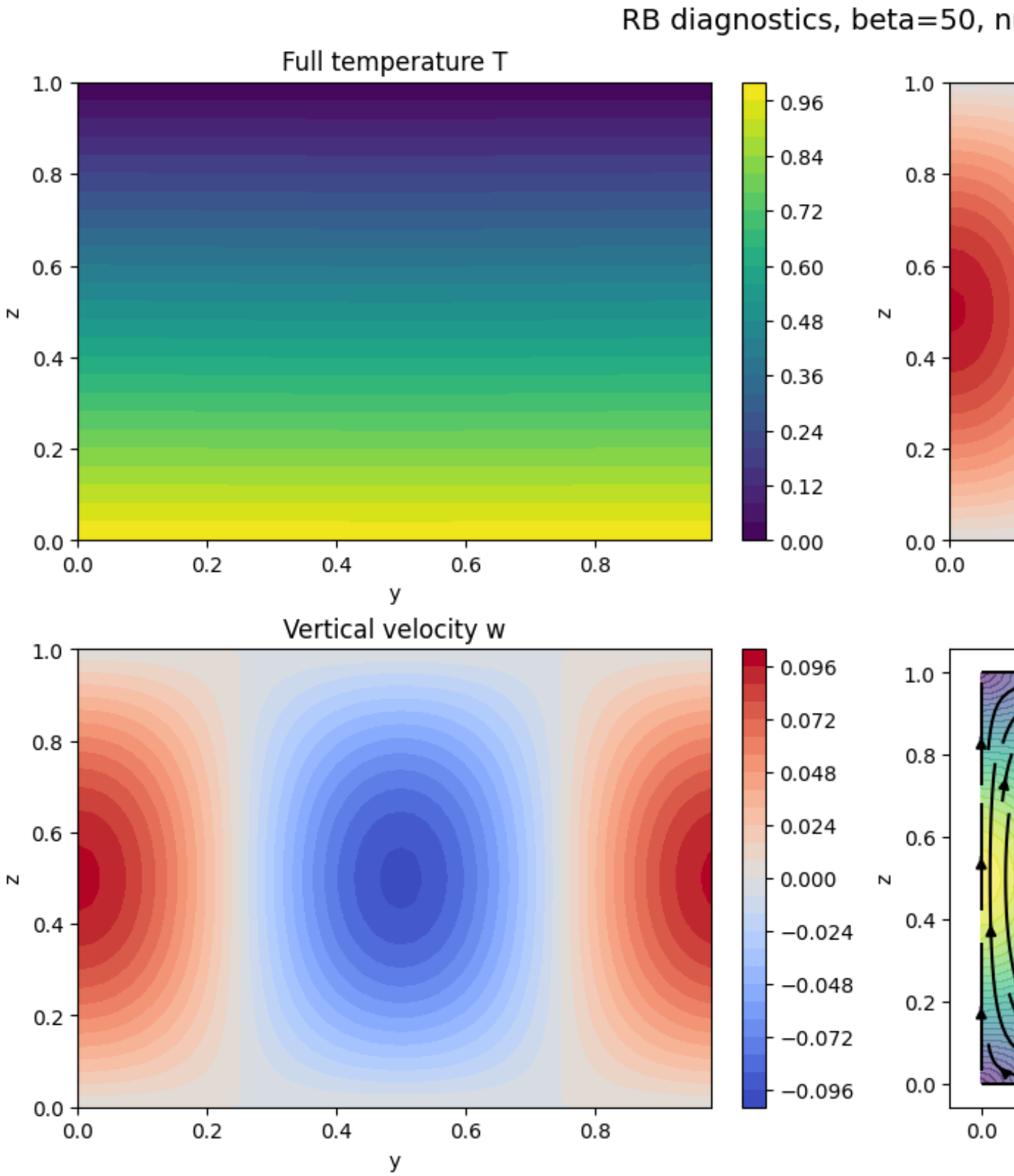


Figure 3.5: Rayleigh-Bénard Diagnostics.

Discuss Results

4 Fazit und Ausblick

(...)

Bibliography

Clymo, R. S. (1970). The Growth of Sphagnum: Methods of Measurement. *Journal of Ecology*, 58(1):13–49. Publisher: [Wiley, British Ecological Society].